Design and Optimization of a Variational Quantum Classifier

- A study on how Quantum Neural Networks (QNN) works

- Alain Abraham

Abstract

This project explores the use of Variational Quantum Circuits (VQCs) for a binary image classification task, specifically distinguishing between horizontal and vertical line patterns. A synthetic dataset containing these patterns is encoded into quantum states using parameterized rotation gates, forming the input to a quantum model.

A trainable quantum circuit (ansatz) is then applied to learn patterns in the data. The model produces outputs by measuring the expectation value of a Pauli-Z observable, resulting in values that can be mapped to class labels.

Training is performed using a hybrid quantum-classical approach, where a classical optimizer (COBYLA or SPSA) updates the circuit parameters to minimize prediction error. During training, non-smooth and non-monotonic loss behavior was observed, highlighting the challenges of optimizing quantum models.

The results show that even small quantum circuits can learn structured patterns and generalize to unseen data, while also emphasizing the importance of circuit design and optimization strategies in Quantum Machine Learning.

Data Encoding

- This is where the process of training our QNN starts.
- Training a Quantum Neural Network requires encoding classical data into a quantum circuit, enabling it to be processed within a quantum system.
- We achieve this using ZFeature Map which encodes our synthetic data into a quantum circuit.

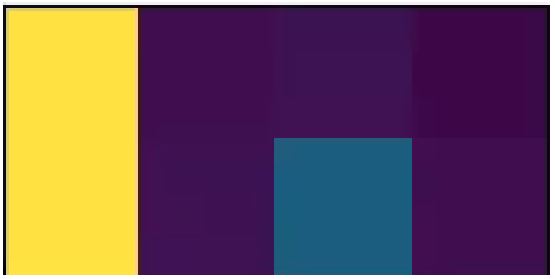


Fig: 1

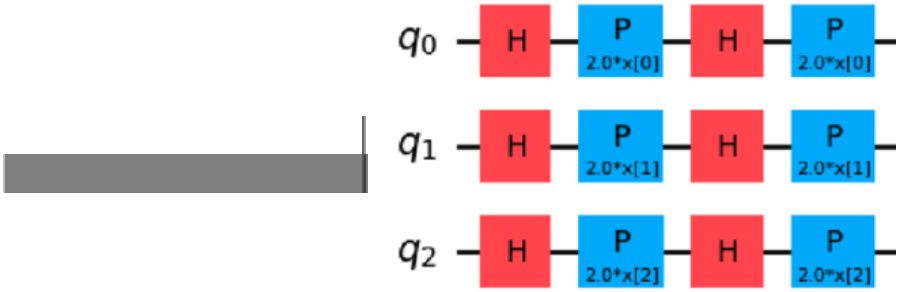


Fig: 2

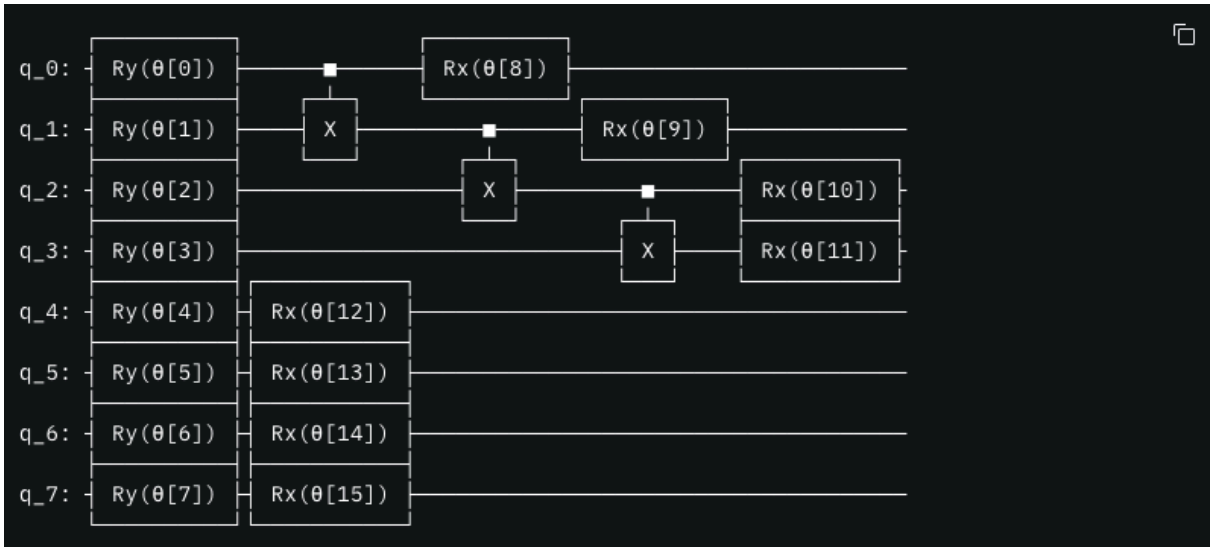
Fig:1 - A sample of our synthetic image containing a vertical line

Fig:2 - The ZFeature map circuit layout

- As we see above our image is converted into a quantum circuit which we will denote using U(x).
- Each qubit in our circuit represents a pixel in our image.
- Since we have an image of 4x2 (4 rows and 2 columns) pixels we will need 8 qubits (0-7).

Creating the Variational Circuit

- The variational circuit is what we call as our ansatz which is a circuit with trainable parameters.
- We will represent this ansatz with W(Θ).
- We will apply two gates Rx and Ry on each qubit which will represent our variational layer and the reason we apply two gates is for more expressivity
- Along with the two variational layers we will also apply CNOT gates for entanglement of the qubits, we do this so that the qubits are able to represent complex correlations in the data.
- The theta value on the Rx and Ry gate are our variable parameters this what makes our circuit variational.

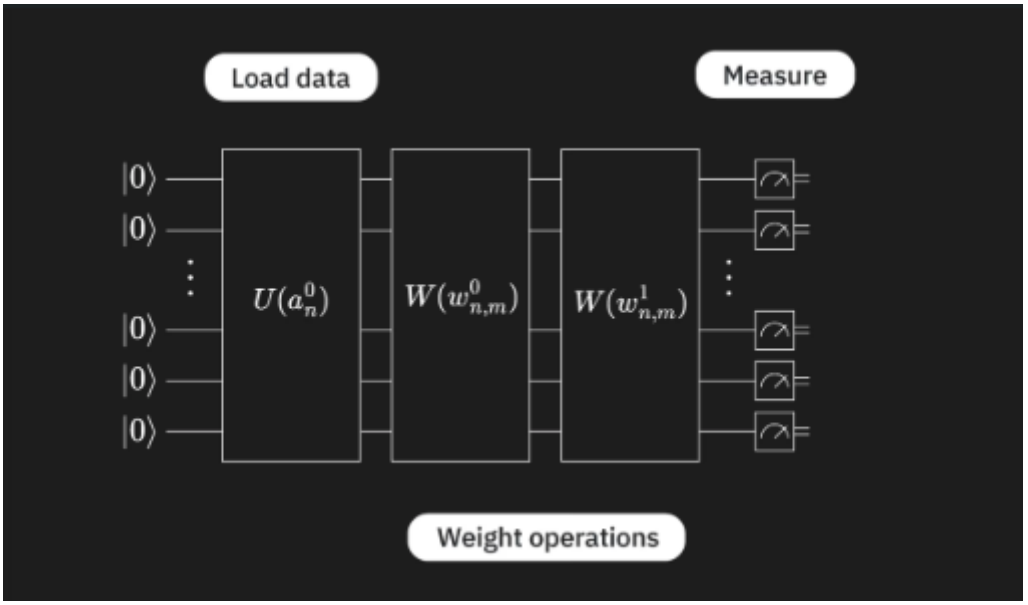


- This is how our variational circuit looks like after applying all the gates

- Depth of the circuit: 3
- The first layer consists of our first variational layer made up of Ry gates
- The second layer consists of our CNOT gates
- And finally the third layer consists of the second variational layer using the Rx gates

Combining both the circuits

- Now that we have completed the data encoding circuit U(x) and our ansatz W(**Θ**) we have to combine both to form a single circuit so that it can be executed on the Quantum processor
- Our combined circuit will have a data encoding circuit and two layers of variational circuits.



The loop of training the QNN

- Now that we have the entire circuit, how do we actually train the QNN?
- Well we start with random or technically motivated parameters for the ansatz, once the parameters are set we run the circuit.
- But how does the circuit classify images and assign labels?
- We have two types of images, either an image with a horizontal line or a vertical line.
- We measure the circuit after the gates are applied, and if we observe the parity of bits returned, if the parity is even then we assign the label to be +1 indicating a vertical line and vice-versa.
- So now that we know how the binary classification is done.
- How do we update the parameters?

- This is where we introduce the **loss function,** once the classification of the data is done we calculate the loss function
- **Loss function:** The loss function is something that tells us how much our predicted classification is relevant with the actual classification, the higher the value of the loss function the more deviance from actual result.
- By calculating the loss function we can know how wrong our prediction was.
- Now we introduce another component called the classical optimizer which takes the loss function as the input and modifies the parameters of the ansatz so that we get a more accurate classification.
- Now we run this loop of Running the circuit -> Performing the binary classification -> calculating the loss function -> Updating the parameter.
- Once the loop converges we take the value of the parameters at the end.

- Now with the parameters we took after the loop converged we run the circuit on unseen data once the prediction is made we compare it with the actual prediction and calculate the accuracy of the model.

Conclusion

Restating the goal:

The project aims to explore the implementation of a Quantum Neural Network (QNN) for processing synthetic image data using quantum feature encoding techniques.

Summary of the approach:

- Encoded the data onto a quantum circuit
- Designed a variational circuit with trainable parameters
- Ran the combined circuit and trained the QNN using the Variational Circuit and Classical Optimizer
- Ran the circuit with parameters and tested the accuracy of the model

Key Insights:

- Demonstrated how quantum circuits can learn patterns

- Highlighted the feasibility of QNN for simple image classification tasks

Future Work:

- The dataset can be scaled to include actual images.
- The model can include deeper circuits for more accurate predictions.
- Include fault tolerant and noise reduction techniques for more accurate readings